

文件资源管理器项目设计文档

- 2351289周慧星

一、项目简介

本项目为同济大学软件学院操作系统课程设计内容，旨在通过实现一个具备真实操作系统文件管理核心功能的虚拟文件资源管理器，加深对文件系统、目录结构、文件管理算法等操作系统知识点的理解和应用。项目采用 C++ 和 Qt 框架，所有操作均作用于内存中的自定义虚拟文件系统（VFS），并支持持久化存储。

程序运行

1. 打开我打包的Release文件夹，找到 `os3-FileSystem.exe`。
2. 点击运行即可，结果会保存至 `data.json`。

二、数据结构设计

1.虚拟文件系统树结构（VNode）

结构定义

```
struct VNode {
    QString name;           // 节点名称
    bool isDir;             // 是否为文件夹
    QString content;        // 文件内容，仅文件有效
    VNode* parent;         // 父节点指针
    QVector<VNode*> children; // 子节点列表
    QDateTime lastModified; // 最后修改时间
    QString type;           // 文件类型（后缀名或“文件夹”）
    ...
};
```

设计说明

- **树形结构**：每个 VNode 代表文件或文件夹，采用 parent-children 树形层级，根节点 parent 为 nullptr。
- **文件夹节点**：isDir=true，content 为空，children 存放所有子节点。
- **文件节点**：isDir=false，content 存放文本内容，children 为空。
- **lastModified**：任何内容/属性修改均更新时间，便于排序和显示。
- **type**：文件用后缀名（如 "TXT 文件"），文件夹为 "文件夹"，便于类型识别和筛选。
- **序列化/反序列化**：通过 toJson()/fromJson() 支持递归地存储和恢复整个 VFS。

典型操作

- 节点的插入/删除/重命名均需维护 parent/children 指针关系，确保树结构完整。
- 递归析构函数保证释放内存不泄漏。

- 每个操作需同步更新时间和类型。

2. 文件系统模型（MVC Model）

VFileSystemModel

- 继承 QAbstractItemModel，负责将 VFS 数据结构映射为 Qt 视图可识别的数据源。
- 支持多列（名称、修改日期、类型）。
- 提供 `nodeFromIndex()` 和 `indexFromNode()`，实现 QModelIndex 与 VNode 指针的高效互查。
- 支持 `refresh()`、`beginModelChange()`、`endModelChange()`，便于批量更新 UI。

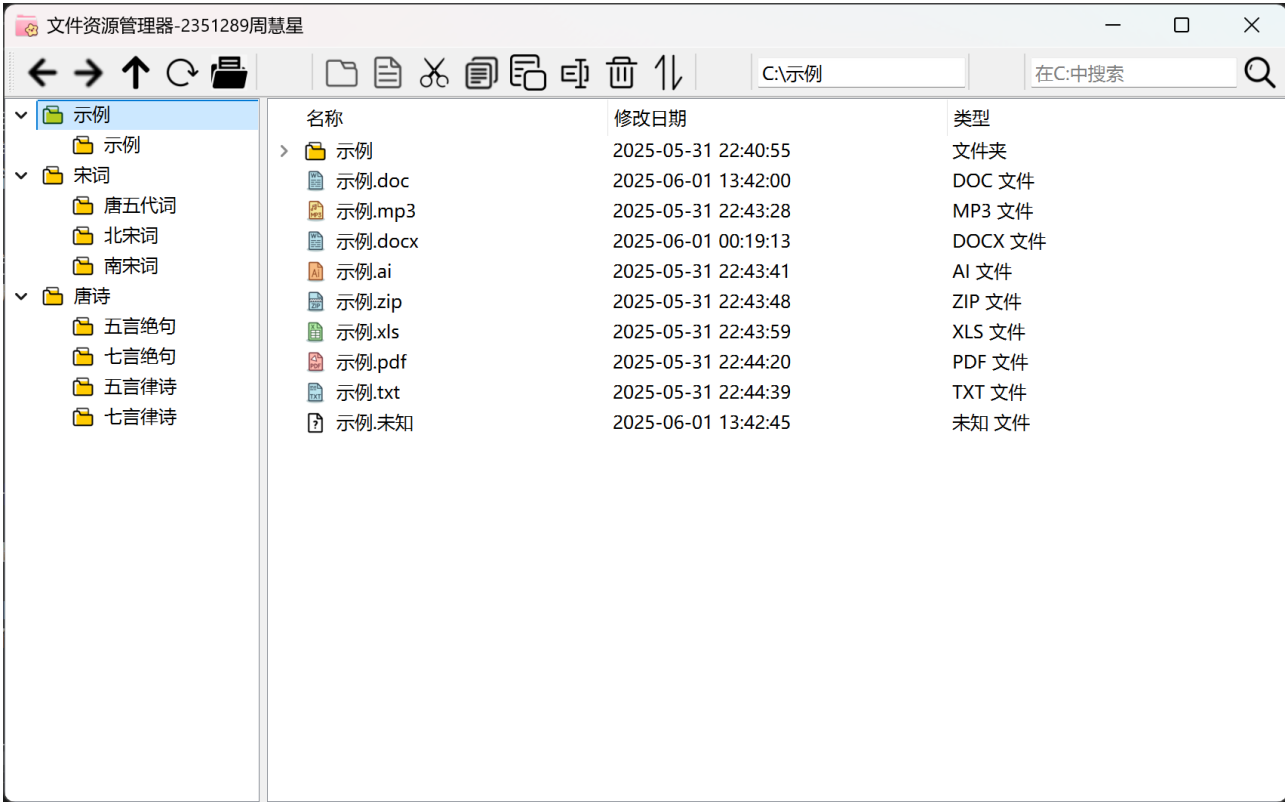
VFolderOnlyModel

- 继承自 VFileSystemModel，重载 `rowCount()` 和 `index()`，只枚举 isDir=true 的节点。
- 用于左侧树形目录，仅展示文件夹，优化性能与用户体验。

三、用户界面与逻辑设计

1. 主界面布局

- **工具栏**：常用操作按钮（新建、删除、重命名、剪切、复制、粘贴、排序、刷新、导航等）。
- **路径栏**：显示当前目录完整路径，便于导航。
- **搜索框**：支持模糊查找所有文件/文件夹。
- **左侧 QTreeView**：基于 VFolderOnlyModel 展示文件夹树，支持单击跳转。
- **右侧 QTreeView**：基于 VFileSystemModel 展示当前目录内容（多列显示、支持多选）。
- **分割器布局**：TreeView 和 ListView 可自由拉伸调整比例。



2. 操作流程与交互逻辑

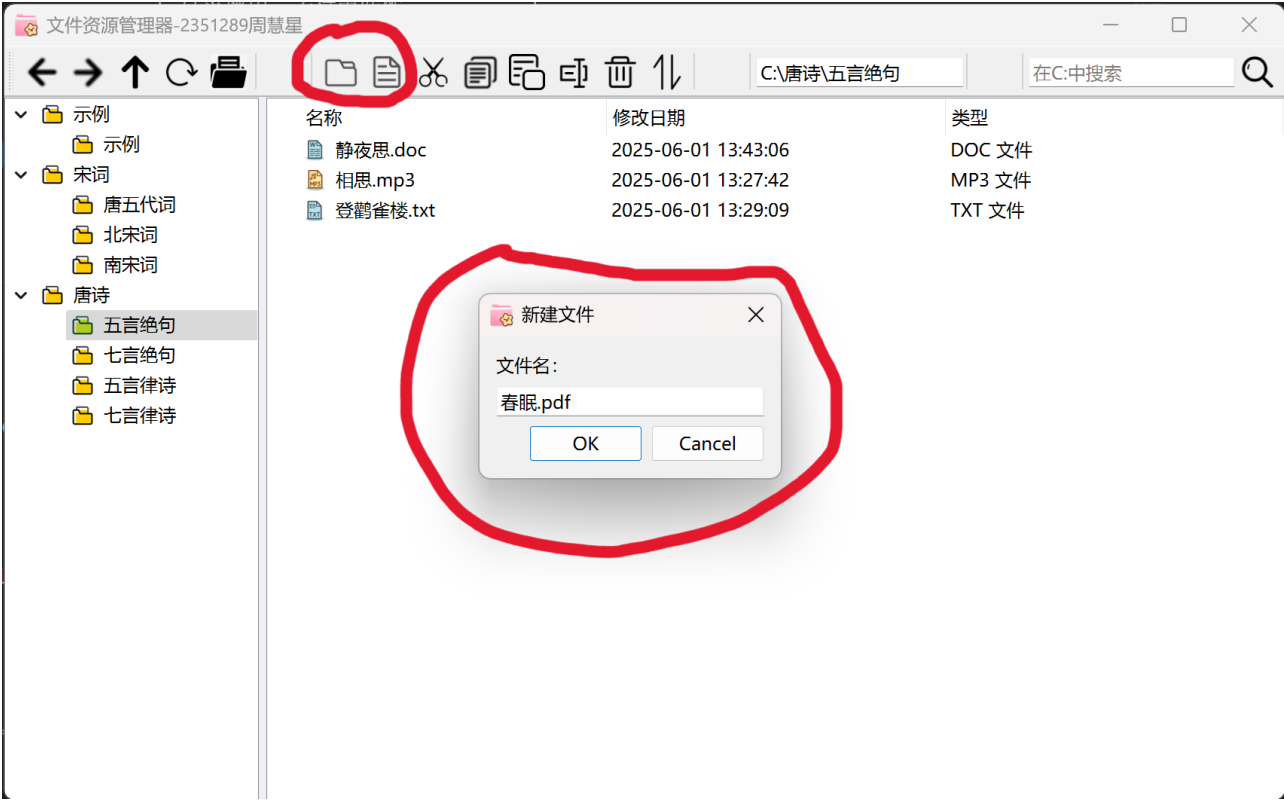
下表展示了主要功能函数及其对应操作系统知识点：

函数名	功能描述	操作系统知识点
onNew(bool isDir)	新建文件或文件夹，防止重名	目录项分配、命名管理
onDelete()	删除选中项，递归释放空间	目录项删除、内存管理
onRename()	重命名，保证同级目录下唯一性	目录项修改、命名约束
onCutOrCopy()	剪切/复制节点，支持多选	目录树节点引用/克隆
onPaste()	粘贴到当前目录，递归移动/复制	目录结构合并、父子关系维护
onSort()	按名称/类型/时间排序	目录项排序、排序算法应用
onSearch()	递归模糊搜索节点	目录遍历、字符串匹配
onOpenFile()	打开/编辑文件内容，自动更新属性	文件内容管理、属性同步
onBack()/onForward()/onUp()	导航历史/上级目录	路径管理、导航栈实现
saveVFS()/loadVFS()	JSON 持久化虚拟文件系统	文件系统元数据持久化

相关辅助函数如 `updateListView()`、`updatePathEdit()`、`recursiveMatch()` 等，保证界面与数据实时同步。

a. 新建文件/文件夹

- 在当前目录下创建新 VNode，弹框获取名称并防止重名，自动更新时间和类型。
- 目录/文件均支持，插入节点后刷新视图。

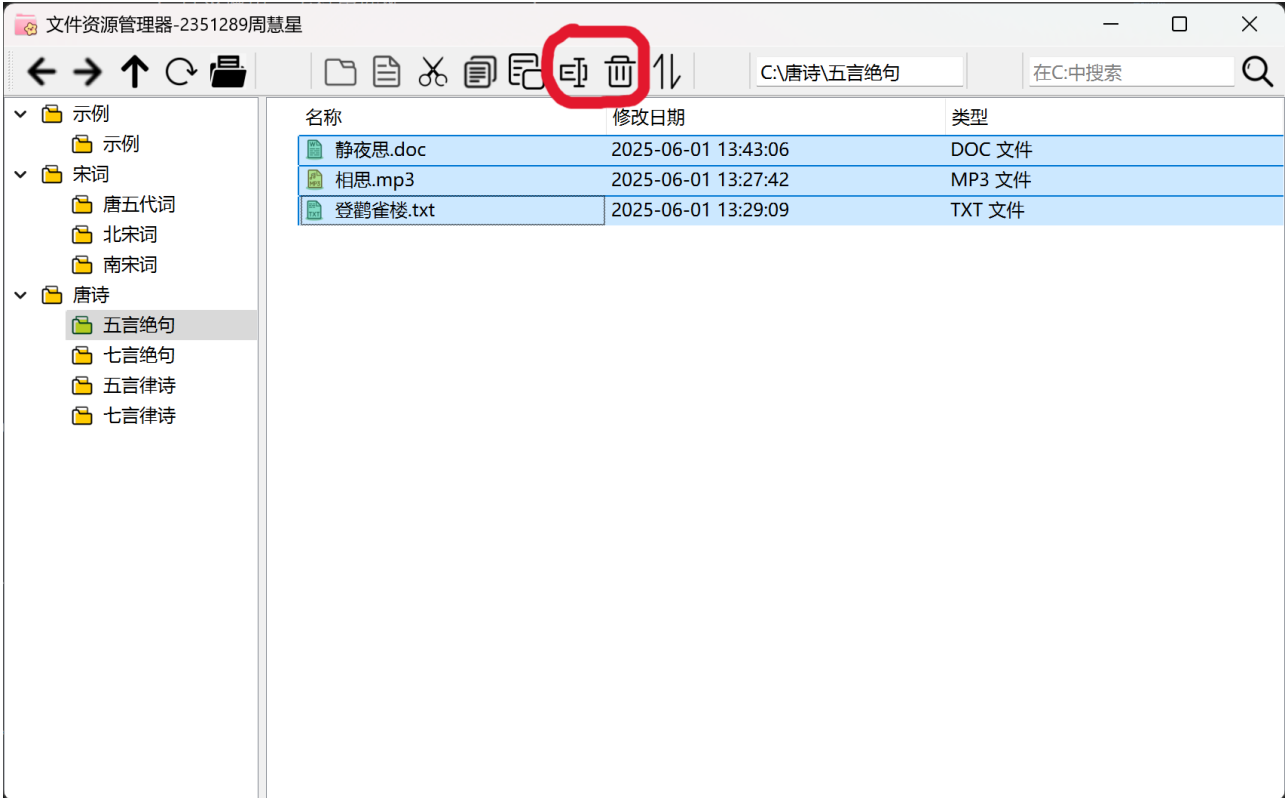


b. 删除

- 支持多选批量删除，删除节点及其所有子孙节点，自动维护 parent/children 链表。
- 同步更新剪贴板、currentIndex 等状态，防止悬挂指针。

c. 重命名

- 多选逐个弹框，防止同级重名，更新 name、type、lastModified。
- 支持取消重命名操作。



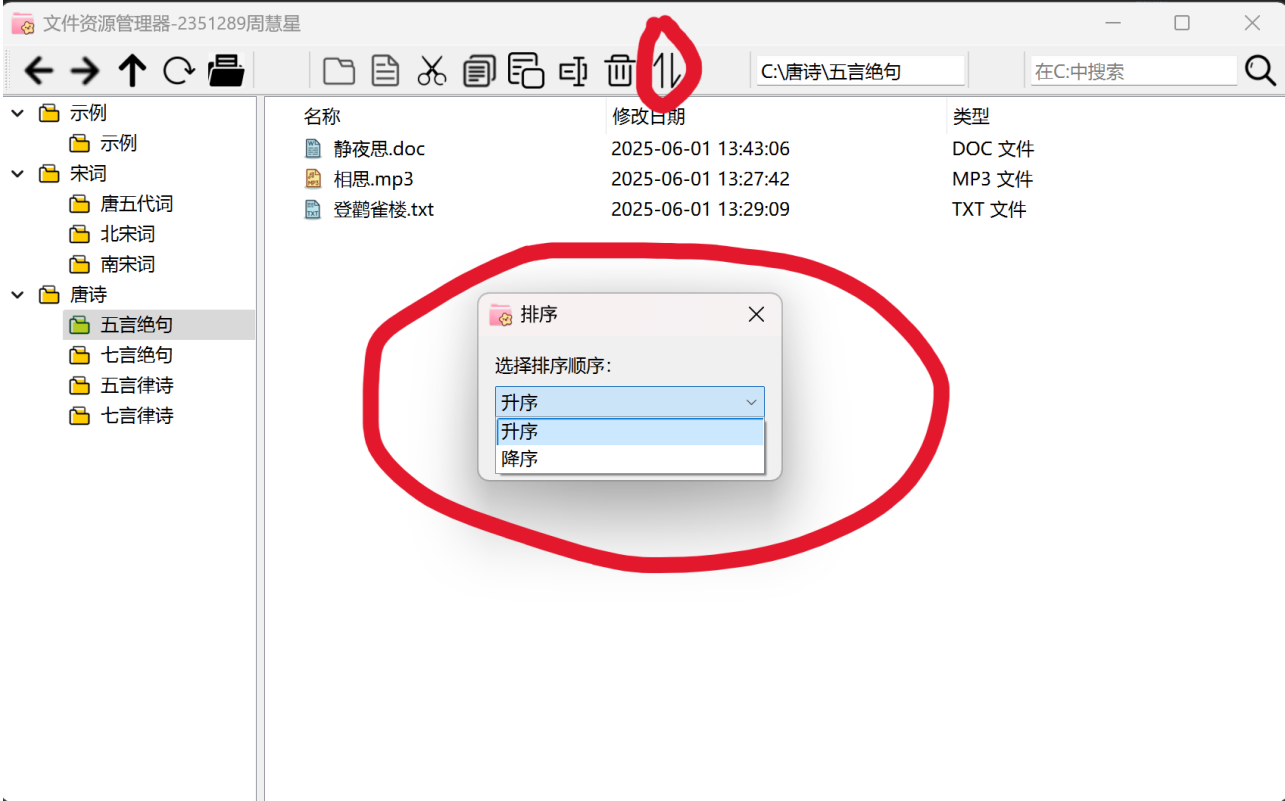
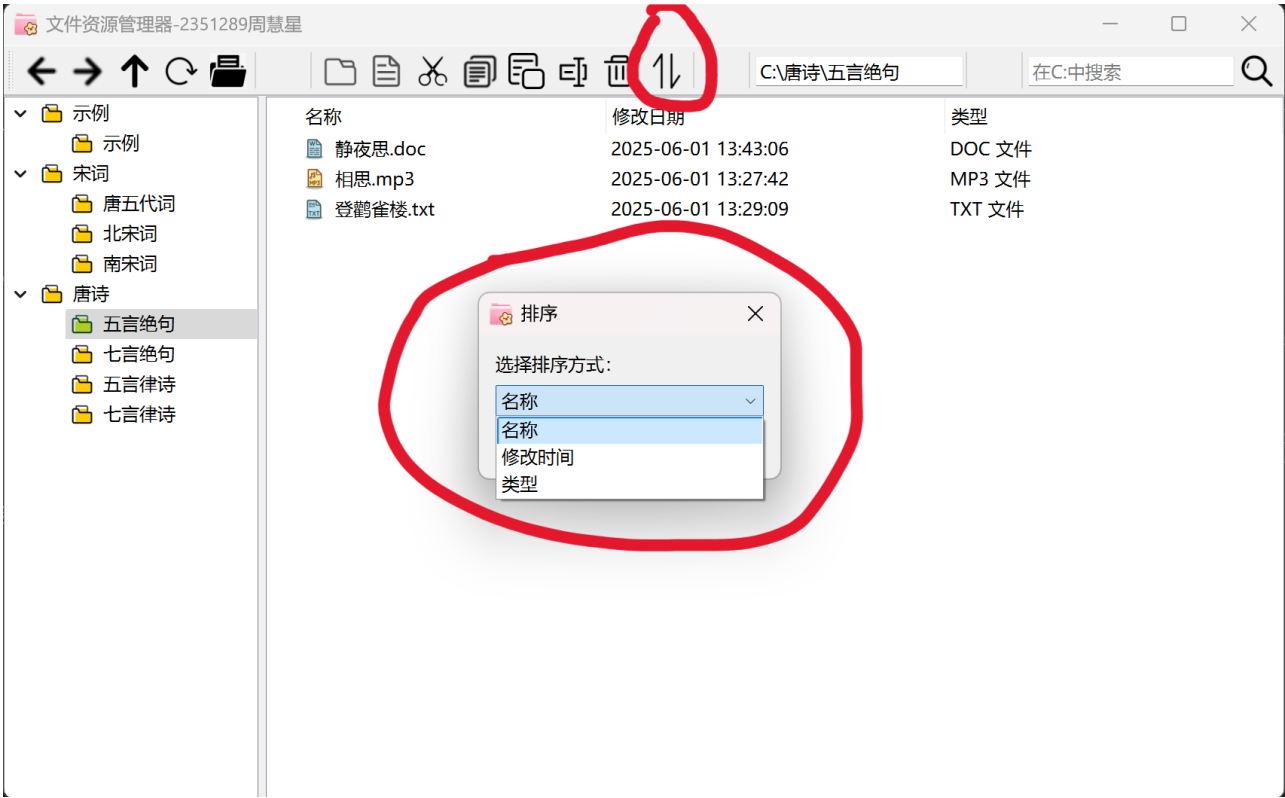
d. 剪切、复制、粘贴

- 剪切：将节点指针保存到剪贴板，粘贴时移动父节点并更新关系。
- 复制：递归 clone 节点与所有子节点，粘贴为新节点。
- 粘贴时自动处理重名冲突（可拓展为自动重命名或覆盖）。
- 剪切后清空剪贴板，粘贴后 isCopy 恢复默认 true。

e. 排序

- 支持按名称、修改时间、类型排序，升序/降序可选。
- 排序通过 std::sort 对当前目录 children 排序，实现高性能排序。

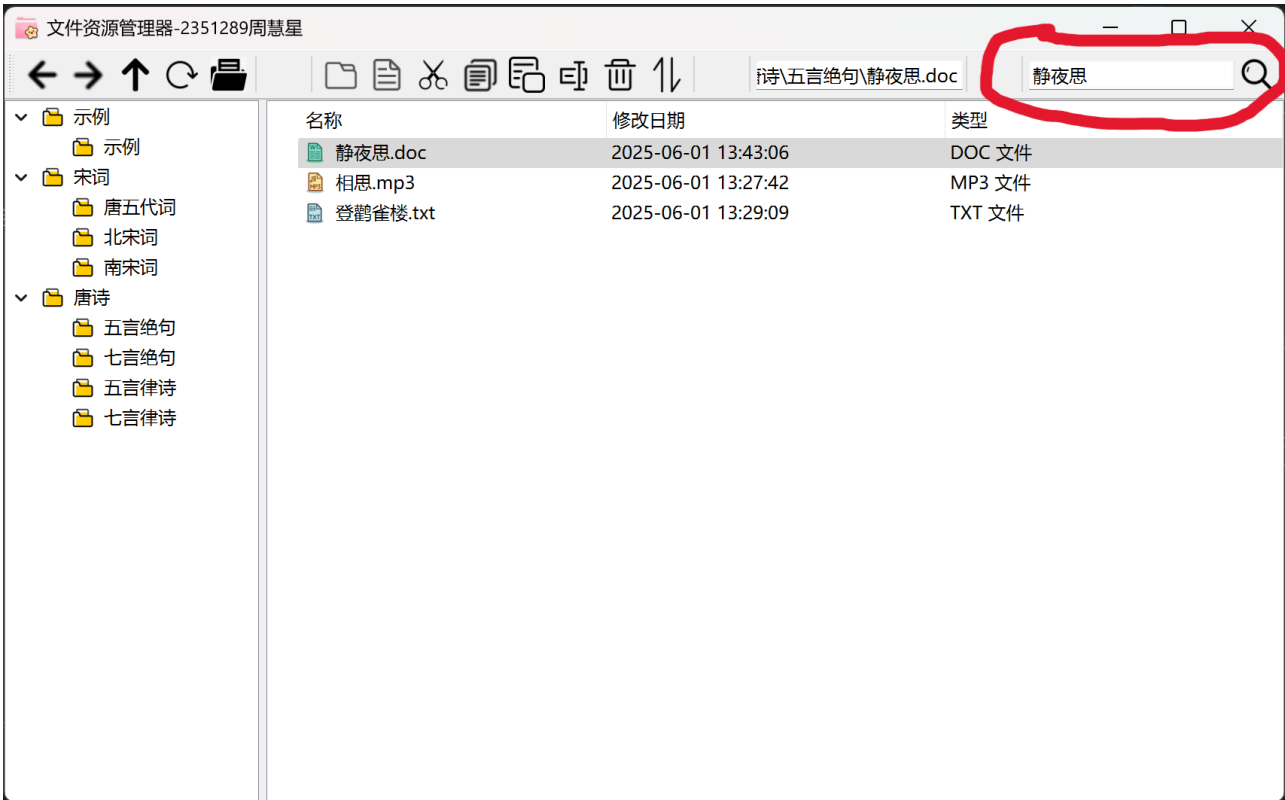
- 排序后刷新视图，保持 UI 一致。



f. 搜索

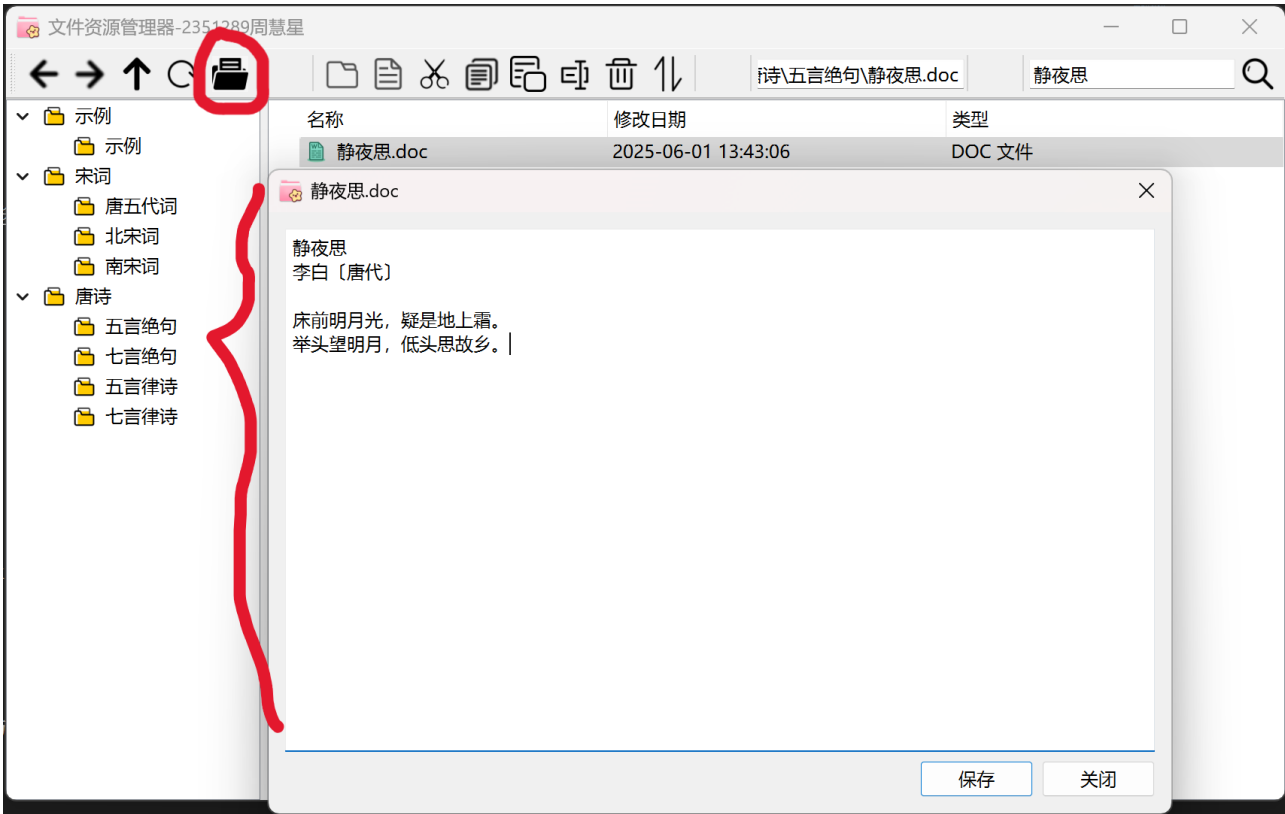
- 递归遍历所有节点，模糊匹配名称，收集所有命中项。
- 支持高亮第一个命中项并跳转，未命中弹窗提示。

- 支持连续搜索、模糊匹配，便于大规模树结构查找。



g. 文件编辑

- 双击文件或通过右键菜单“打开”触发编辑。
- 弹出 FileEditDialog，基于 QTextEdit 编辑内容，支持保存、关闭时保存提示。
- 内容保存后更新时间，刷新当前目录视图。

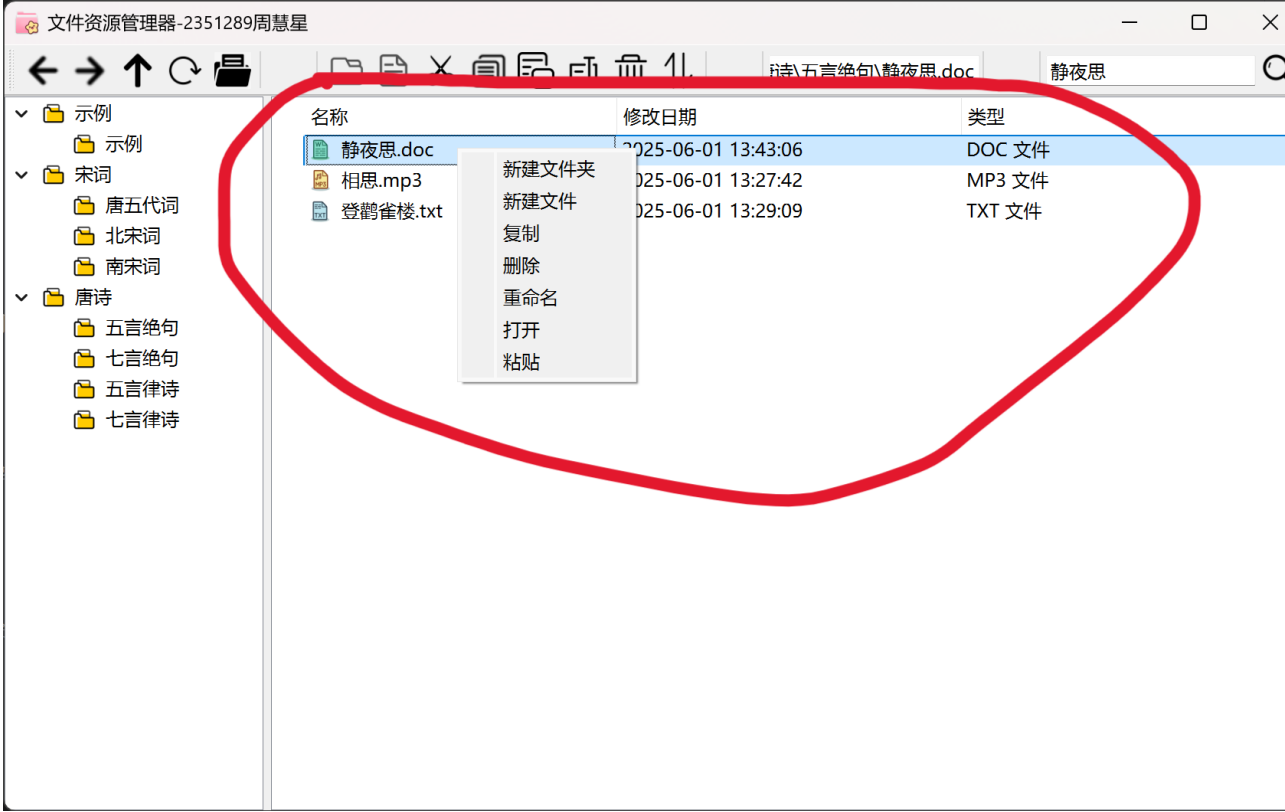


h. 导航

- 后退、前进、上级目录功能基于 QStack 实现，保证操作可逆与历史导航。
- 路径栏可视化显示当前位置，便于用户定位。

i. 右键菜单

- 支持在 ListView 上右键弹出常用操作菜单（新建、复制、删除、重命名、打开、粘贴等）。
- 根据选中项类型动态调整菜单项（如文件才显示“打开”）。



j. 数据持久化

- 关闭程序时自动将 VFS 序列化为 data.json。
- 启动时自动检测并加载 data.json，若无则初始化空根目录。
- 采用 QJsonDocument/QJsonObject/QJsonArray 实现递归存储与恢复。

四、模块与文件划分

- **main.cpp** 程序入口，初始化 QApplication 和 MainWindow。
- **mainwindow.h/cpp** 主窗口，处理 UI、信号槽和各类业务逻辑。
- **vfilesystem.h** VNode 结构定义及序列化方法。
- **vfilesystemmodel.h/cpp** VFileSystemModel/VFolderOnlyModel，Qt Model 层实现。
- **fileeditdialog.h** 文件编辑弹窗，处理文本文件内容编辑和保存。
- **icons/** 图标资源，按类型分配。
- **data.json** 虚拟文件系统的本地持久化文件（自动生成）。

五、关键严谨性与鲁棒性设计

- **指针管理**：VFS 树结构所有节点析构均采用递归 delete，防止内存泄漏。

- **多模型索引转换**：TreeView 与 ListView 分别用不同 Model，所有 QModelIndex 交互均通过 VNode 指针转换，严格保证一致性与安全性。
- **异常判断**：所有操作均判断节点有效性（如 parent 是否为空、重名检查、剪贴板合法性等），防止越界和崩溃。
- **界面刷新**：任何数据变更均及时刷新 UI，确保数据与视图同步。
- **持久化安全**：保存/加载均使用标准 Qt JSON API，支持复杂树结构递归序列化，防止数据丢失或格式错误。
- **多线程安全**：项目当前所有操作均在主线程 UI 线程完成，避免竞态问题。可扩展时考虑多线程访问保护。

六、设计流程图（伪代码示意）

```
graph TD
    MainWindow -->|操作| VFileSystemModel
    VFileSystemModel -->|数据| VNode
    MainWindow -->|编辑| FileEditDialog
    MainWindow -->|持久化| data.json
    VNode -->|序列化| data.json
    VFileSystemModel -->|树/列表| QTreeView
    MainWindow -->|信号槽| QToolBar
```

七、操作系统理论知识应用总结

1. **多级目录树结构**：完整模拟了实际操作系统中的目录树，实现了节点递归管理、树形遍历、节点父子关系维护等。
2. **目录项管理与元数据维护**：所有文件/文件夹均有独立元数据，支持属性自动更新和约束检查。
3. **文件内容与编辑**：通过内存操作模拟实际文件内容读写，实现了简易的内容管理与数据同步。
4. **文件操作算法**：批量操作、递归删除/复制/移动、排序/搜索等功能，均对应操作系统课程的文件系统管理算法。
5. **数据持久化与恢复**：仿照文件系统元数据管理，采用 JSON 序列化方式保存与恢复虚拟文件系统结构。